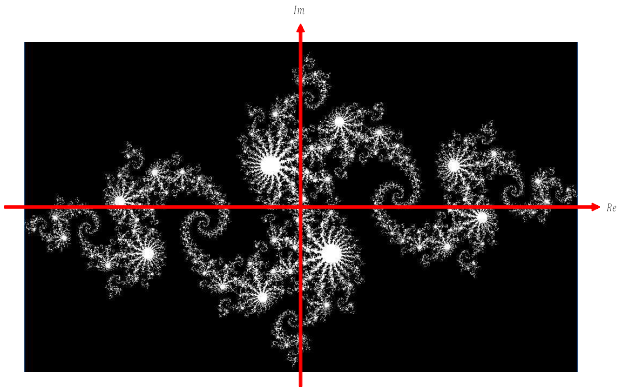

실습 (Julia Set 이미지 만들기)

윤 승 현
동국대학교 컴퓨터·AI학부

학습 내용

- 복소 평면 위의 Julia set의 정의
- Julia set을 구현
 - CPU 구현
 - GPU 구현

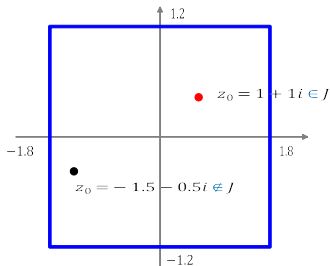


Julia Set

- 복소수 $z_0 = a + bi$ 에 대한 점화식

$$z_{n+1} = z_n^2 + c, \quad n = 0, 1, \dots, N(= 256)$$

- 여기서 $c = -0.8 + 0.156i$: 임의 복소수
- Julia 집합 $J := \{z_0 \mid |z_n| < 2, n = 1, 2, \dots, N\}$



Julia Set (CPU)

```
#include <iostream>
#include <fstream>
#include <complex>
#include <vector>
#include <cmath>

int main()
{
    const int width = 1920;
    const int height = 1080;
    const int maxIter = 300;

    // Julia set 파라미터:  $z_{n+1} = z_n^2 + c$ 
    const std::complex<double> c(-0.8, 0.156);

    // 복소 평면 범위
    const double xmin = -1.8;
    const double xmax = 1.8;
    const double ymin = -1.2;
    const double ymax = 1.2;
    const double R = 2.0;
    std::vector<unsigned char> image(width * height * 3);
```

Julia Set (CPU)

```
for (int j = 0; j < height; ++j) {
    for (int i = 0; i < width; ++i) {
        double x = xmin + (xmax - xmin) * i / (width - 1);
        double y = ymax - (ymax - ymin) * j / (height - 1);
        std::complex<double> z(x, y); // 픽셀 (i, j) -> 복소 평면  $z = x + yi$ 

        int iter = 0;
        while (std::abs(z) <= R && iter < maxIter) {
            z = z * z + c;
            ++iter;
        }

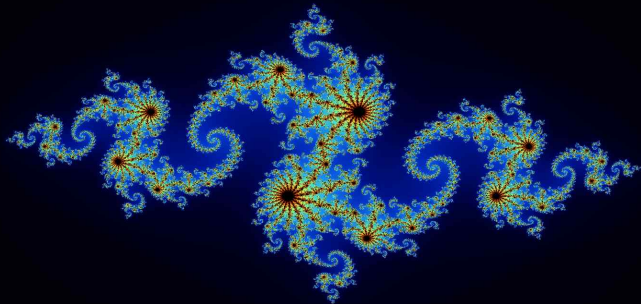
        int idx = 3 * (j * width + i);
        if (iter == maxIter) { //  $z$ 가 Julia set에 포함
            image[idx + 0] = image[idx + 1] = image[idx + 2] = 0;
        }
        else { //  $z$ 가 Julia set에 미포함
            double t = (double)iter / maxIter; //  $0 < t < 1.0$ 
            image[idx + 0] = (unsigned char)(9 * (1 - t) * t * t * t * 255);
            image[idx + 1] = (unsigned char)(15 * (1 - t) * (1 - t) * t * t * 255);
            image[idx + 2] = (unsigned char)(8.5 * (1 - t) * (1 - t) * (1 - t) * t * 255);
        }
    }
}
```

Julia Set (CPU)

```
// Save as binary PPM (P6)
std::ofstream ofs("julia.ppm", std::ios::binary);
ofs << "P6\n" << width << " " << height << "\n255\n";
ofs.write(reinterpret_cast<const char*>(image.data()), image.size());
ofs.close();
std::cout << "Saved julia.ppm\n";
return 0;
}
```

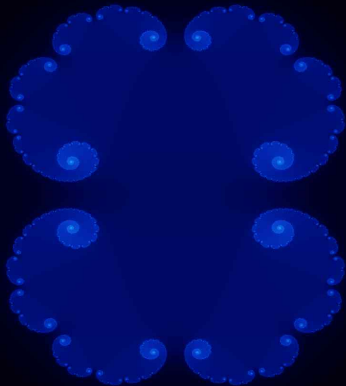
Julia Set 결과

$$c = -0.8 + 0.156i$$



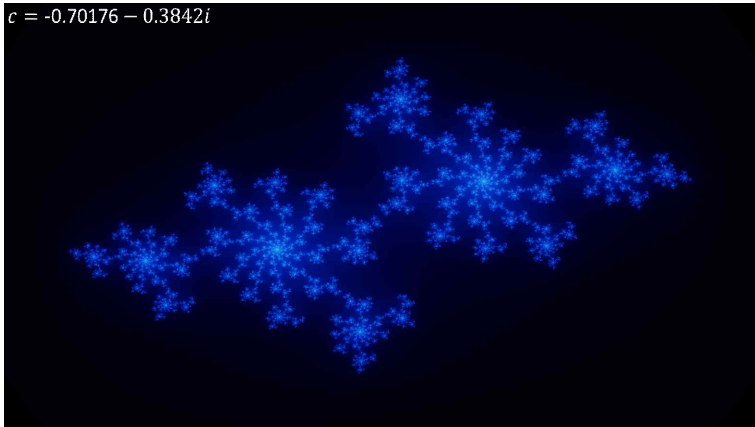
Julia Set 결과

$c = 0.285 + 0i$



Julia Set 결과

$c = -0.70176 - 0.3842i$



실습: Julia Set (GPU)

- 다음의 프로토 타입을 갖는 GPU 커널을 사용하여 프로그램을 작성

```
__global__ void juliaKernel(  
    unsigned char* image,  
    int width, int height,  
    int maxIter,  
    float xmin, float xmax, float ymin, float ymax,  
    float cx, float cy, float R)
```

```
dim3 block(16, 16);  
dim3 grid((width + 15) / 16, (height + 15) / 16);  
auto total_start = std::chrono::high_resolution_clock::now();  
{  
    juliaKernel << <grid, block >> > (d_image, width, height, maxIter, xmin, xmax, ymin, ymax, cx, cy, R);  
    cudaDeviceSynchronize();  
    cudaMemcpy(h_image.data(), d_image, imageSize, cudaMemcpyDeviceToHost);  
}  
auto total_end = std::chrono::high_resolution_clock::now();  
std::chrono::duration<double, std::milli> total_ms = total_end - total_start;  
std::cout << "Total time: " << total_ms.count() << " ms\n";
```

Q&A